

حالة التقرير: نسخة التسليم والمناقشة الأكاديمية الرسمية

إعداد الطالب: مشعل

نتائج الاختبارات: 33 اختباراً ناجحاً من أصل 33 (JVM %100)

ملف التطبيق: app/build/outputs/apk/debug/app-debug.apk

1. الملخص التنفيذي

مشروع **LOCAL CONTACT** هو تطبيق اتصال مباشر بين أجهزة Android عبر الشبكة المحلية، مثل Wi-Fi أو Hotspot، دون الاعتماد على خادم مركزي أو خدمة سحابية. قبل بدء عملية التحسين، كان التطبيق يحتوي على وظائف أساسية تعمل، لكنه كان يحتاج إلى تنظيم هندسي أوضح، وربط رسمي بين المتطلبات والكود والاختبارات، ومعالجة أعمق لمشكلة إدارة مفاتيح التشفير.

تم تطوير المشروع على مسارين متوازيين. المسار الأول هو **هندسة البرمجيات**، وشمل توثيق المتطلبات، وتحديد خط الأساس، وإنشاء مصفوفة تتبع، وفصل إدارة المفاتيح عن محرك التشفير، وإضافة اختبارات منظمة. المسار الثاني هو **التشفير وأمن الاتصال**، وشمل الحفاظ على AES-256-GCM، وإضافة مكونات ECDH و HKDF لمفاتيح الجلسات، وإضافة هوية جهاز عبر Android Keystore، وإضافة توقيع مرتبط بسياق المصافحة، وتثبيت هوية القرين عبر TOFU.

نجحت اختبارات JVM المسجلة في المشروع بنتيجة **33 اختباراً ناجحاً من أصل 33**. كما تم بناء نسخة Debug بنجاح وإنشاء ملف APK. مع ذلك، لا يمثل ذلك تحققاً أمنياً ميدانياً كاملاً؛ إذ لم يُنفذ اختبار اتصال فعلي بين جهازين، ولم تُنفذ SAS، وما زالت Replay Protection الكاملة و File Integrity Checksum غير منفذتين، كما أن بعض مسارات الغرف والملفات والوسائط تستخدم المفتاح الشبكي القديم.

⚠️ النتيجة النهائية المعتمدة:

أصبح المشروع جاهزاً للتسليم والمناقشة الأكاديمية بوصفه **نسخة موثقة ومختبرة جزئياً**، لكنه لا يُقدم على أنه نظام أمني نهائي تم التحقق منه ميدانياً في جميع مساراته.

الجزء الأول: هندسة البرمجيات

2. حالة المشروع قبل تطبيق مبادئ هندسة البرمجيات

قبل التحسين، كان التركيز الأساسي على وجود الوظائف داخل التطبيق، مثل الاتصال المحلي، والرسائل، ونقل الملفات، والمكالمات، والتشفير. كان الكود يحتوي على مكونات مهمة، إلا أن الصورة الهندسية الكاملة لم تكن موحدة في وثائق رسمية.

كانت بعض المسؤوليات متقاربة داخل مكونات كبيرة، وخصوصاً محرك الاتصال ومحرك التشفير. كما لم يكن من السهل الإجابة بدقة عن الأسئلة التالية: ما المتطلبات الرسمية؟ أين يطبق كل مطلب؟ ما الاختبار الذي يثبت تنفيذه؟ هل الميزة منفذة بالكامل أم جزئياً؟ وهل النتيجة مثبتة على JVM أم على جهاز Android حقيقي؟

لم يكن هذا يعني أن التطبيق لا يعمل، بل يعني أن قابلية الصيانة والتتبع والإثبات الأكاديمي كانت أضعف من المطلوب. كان من الممكن أن توجد وظيفة في الكود دون وجود وثيقة تربطها بمتطلب أو اختبار أو معيار قبول.

3. ما تم تطبيقه في هندسة البرمجيات

3.1 وثيقة مواصفات المتطلبات SRS

تم إنشاء الملف:

docs/SRS.md

يمثل هذا الملف وثيقة **Software Requirements Specification**. وهو يصف المتطلبات الوظيفية وغير الوظيفية، وحالات الاستخدام، والحالات الحدية، ومعايير القبول، والقيود العامة للنظام. أصبح من الممكن ربط وظائف مثل الرسائل والملفات والاتصال المحلي والتشفير بمتطلبات مكتوبة بدل تركها موزعة في الشرح الشفهي أو داخل الكود فقط.

3.2 وثيقة خط الأساس Baseline

تم إنشاء الملف:

يوثق هذا الملف حالة المشروع في نقطة مرجعية محددة. ويشرح البنية العامة، والمكونات، وتدفقات الشبكة، والتشفير الموجود، والاختبارات، والفجوات المعروفة. أهمية خط الأساس أنه يوضح الفرق بين حالة المشروع قبل التعديلات وحالته بعد التعديلات، ويمنع الخلط بين ميزة كانت موجودة مسبقًا وميزة تمت إضافتها خلال عملية التحسين.

3.3 مصفوفة التتبع RTM

تم إنشاء الملف:

docs/TraceabilityMatrix.md

تربط مصفوفة التتبع بين المتطلب والكود والاختبار وحالة التحقق. أصبح المسار الهندسي للمعلومة كما يلي:

Requirement (المتطلب)	← Source Code (الكود)	← Test Evidence (دليل الفحص)	← Current Status (الحالة)
-----------------------	-----------------------	------------------------------	---------------------------

وتوضح المصفوفة الفرق بين الحالات التالية:

- منفذ ومختبر على JVM.
- مدمج في الكود لكنه غير مختبر بين جهازين.
- منفذ جزئيًا.
- غير منفذ.
- مقترح للتحسين المستقبلي.

هذه الدقة مهمة في المناقشة لأنها تمنع تحويل وجود كود أو اختبار جزئي إلى ادعاء بأن النظام الكامل تم التحقق منه.

3.4 فصل إدارة المفاتيح عبر KeyProvider

قبل التحسين، كان `LocalCryptoEngine` مرتبطًا مباشرة بمنطق إنشاء المفتاح المشتق من عبارة المرور. تم إنشاء واجهة:

KeyProvider

وتطبيق يحافظ على السلوك القديم:

LegacyStaticKeyProvider

أصبح التدفق المعماري كالتالي:

LocalCryptoEngine ↓ KeyProvider ↓ LegacyStaticKeyProvider ↓ PBKDF2 ↓ AES Network Key

هذا التغيير لم يُلغِ المفتاح الثابت في المرحلة الأولى، لكنه فصل المسؤولية وأصبح من الممكن إضافة مزود مفاتيح آخر لاحقًا دون إعادة كتابة كل دوال التشفير. كما حافظت الاختبارات الحالية على سلوك التشفير القديم.

3.5 تنظيم الاختبارات

تم تنظيم اختبارات مستقلة لمكونات مختلفة، منها:

مجموعة الاختبارات	الغرض	النتيجة المسجلة
LocalCryptoEngineTest	اختبار AES-GCM، التلاعب، المفتاح الخاطئ، JSON والكتل	10/10
EcdhEngineTest	اختبار اتفاق الطرفين على السر المشترك واشتقاق مفاتيح الجلسات	8/8
PeerSessionKeyManagerTest	اختبار إدارة مفاتيح الجلسات والحالات	4/4
HandshakeHardeningTest	اختبار التوقيع المرتبط بالسياق و TOFU ومنع بعض حالات التراجع	8/8
اختبارات المشروع الأخرى	اختبارات إضافية موجودة في المشروع	3/3
إجمالي النتيجة المسجلة:		33 / 33

33 tests executed 33 passed 0 failed 0 skipped

هذه النتيجة تخص اختبارات JVM التي تم تشغيلها، ولا تعني أن جميع متطلبات التطبيق أو جميع مسارات الشبكة تم اختبارها.

4. مقارنة هندسة البرمجيات قبل وبعد

المجال	قبل التحسين	بعد التحسين
المتطلبات	موزعة وغير مرتبطة دائماً بالكود والاختبار	موثقة في <code>SRS.md</code>
الحالة المرجعية	غير موحدة	موثقة في <code>Baseline.md</code>
تتبع المتطلبات	محدود أو غير صريح	موثق في <code>TraceabilityMatrix.md</code>
إدارة المفاتيح	مرتبطة مباشرة بمحرك التشفير	معزولة خلف <code>KeyProvider</code>
الاختبارات	لم تكن تغطي الصورة الأكاديمية بوضوح	اختبارات منفصلة للتشفير و ECDH و Handshake
تقييم الإنجاز	قد يخلط بين وجود الكود والتحقق الفعلي	حالات دقيقة: JVM Tested, Pending, Missing
إدارة المخاطر	غير موثقة بصورة كافية	مخاطر MITM و Downgrade و Replay موثقة
قابلية التغيير	إضافة Keystore أو Session Keys قد تتطلب تعديلاً واسعاً	يمكن إضافة مزودات مفاتيح ومكونات مستقلة تدريجياً
الأدلة الأكاديمية	تعتمد على الشرح الشفهي	وثائق، اختبارات، ومسارات تتبع
قابلية المناقشة	صعبة الإثبات	يمكن شرح القرار والدليل والحدود بوضوح

5. الأثر الهندسي للتحسين

لم يكن الهدف من هندسة البرمجيات مجرد إضافة ملفات توثيق، بل تحويل المشروع من مجموعة وظائف تعمل إلى نظام يمكن تفسيره وإدارته وتقييمه. أصبحت كل نتيجة مهمة مرتبطة بدليل. فإذا كانت الميزة مختبرة على JVM، نذكر ذلك. وإذا كانت مدمجة في الكود دون اختبار ميداني، نذكر ذلك. وإذا كانت غير منفذة، نذكر أنها غير منفذة بدل تقديمها على أنها مكتملة.

هذا الأسلوب يرفع جودة المشروع أمام أستاذ هندسة البرمجيات، لأنه يوضح دورة المتطلب والتصميم والتنفيذ والاختبار والتتبع وإدارة المخاطر.

الجزء الثاني: التشفير وأمن الاتصال

6. حالة التشفير قبل التحسين

كان التطبيق يستخدم تشفيراً متماثلاً من النوع:

`AES/GCM/NoPadding`

بحجم مفتاح:

`AES-256 = 256 bits`

وكان المفتاح مشتقاً من عبارة مرور ثابتة داخل الكود، مع استخدام PBKDF2-HMAC-SHA256 وملح ثابت وفق التنفيذ الموجود في المشروع.

كان التدفق الأمني القديم كما يلي:

الرسائل وبعض بيانات الشبكة ↓ AES-256-GCM ↓ مفتاح ثابت مشترك ↓ PBKDF2WithHmacSHA256 ↓ DEFAULT_PASSPHRASE

كانت الحزم تستخدم Nonce عشوائياً بطول 12 بايت في بعض التدفقات. وكانت كتل الملفات تستخدم Nonce مشتقاً من معرف الملف ورقم الكتلة ومعرف الجلسة حسب التنفيذ الموجود.

7. مشكلات التصميم القديم

كان المفتاح الثابت المشترك يمثل نقطة ضعف معمارية. إذا تم استخراج عبارة المرور من التطبيق أو تحليل ملف APK، يستطيع المهاجم اشتقاق المفتاح نفسه المستخدم لدى الأجهزة والجلسات.

كما لم يكن هناك تبادل مفاتيح ديناميكي بين الأجهزة. ولم تكن هناك هوية تشفيرية مستقلة لكل جهاز. ولم تكن هناك حماية كاملة من إعادة بث حزمة صحيحة قديمة. كما لم يكن هناك تحقق SHA-256 للملف كاملاً بعد اكتمال نقله.

يجب التمييز بين قوة AES-GCM كخوارزمية وبين طريقة إدارة المفتاح. قد تكون الخوارزمية قوية، لكن استخدام مفتاح ثابت مكشوف يضعف النظام ككل.

8. ما تم تطبيقه في التشفير

8.1 الحفاظ على AES-256-GCM

لم يتم استبدال AES-GCM. بقيت الخوارزمية الأساسية لتشفير البيانات لأنها توفر السرية وسلامة البيانات عبر Authentication Tag.

حسب التنفيذ الموثق، يستخدم النظام:

- مفتاح AES بطول 256 بت.
- GCM Authentication Tag بطول 128 بت.
- Nonce بطول 12 بايت في مسارات الحزم.
- رفض فك التشفير عند تعديل النص المشفر أو وسم المصادقة.

8.2 اختبارات محرك التشفير

اختبارات LocalCryptoEngineTest تحققت من السلوك الخوارزمي لمحرك التشفير، بما في ذلك:

- نجاح التشفير ثم فك التشفير.
- رفض المفتاح الخاطئ.
- رفض النص المشفر بعد التلاعب.
- رفض Authentication Tag بعد التلاعب.
- تشفير وفك تشفير JSON.
- التعامل مع النصوص العربية و UTF-8.
- اشتقاق Nonce للكتل.
- تشفير وفك كتل البيانات.
- بعض الحالات الحدية.

نجاح هذه الاختبارات يثبت صحة السلوك الذي تختبره، لكنه لا يثبت أمان النظام الشبكي الكامل.

8.3 ECDH و HKDF لمفاتيح الجلسات

تم إنشاء مكون ECDH لتوليد مفاتيح مؤقتة وحساب Shared Secret بين طرفين، ثم اشتقاق Session Key باستخدام HKDF-SHA256 وفق التصميم المنفذ. التدفق المنطقي هو:

Ephemeral Private Key A + Peer Public Key B ↓ Shared Secret A ↓ HKDF-SHA256 + Session Context ↓ Session Key A

ويحسب الطرف الآخر السر نفسه بطريقة عكسية:

Ephemeral Private Key B + Peer Public Key A ↓ Shared Secret B ↓ HKDF-SHA256 + نفس السياق ↓ Session Key B

ينبغي اعتماد خوارزمية المنحنى الفعلية من الكود النهائي عند التسليم. التقارير المرحلية أشارت إلى P-256 في بعض المراحل، بينما وردت إشارة إلى X25519 في ملخص لاحق؛ لذلك يجب عدم ذكر X25519 في المناقشة إلا إذا كان ملف EcdhEngine.kt يثبت ذلك صراحة. الثابت المؤكد من الاختبارات هو وجود مكون ECDH ونجاح اختبارات اتفاق السر واشتقاق مفتاح الجلسة.

8.4 Android Keystore وهوية الجهاز

تم إنشاء KeystoreIdentityManager لاستخدام Android Keystore في إدارة مفتاح هوية الجهاز. الهدف هو الاحتفاظ بالمفتاح الخاص داخل مخزن المفاتيح وعدم وضعه في Room أو ملف عادي أو SharedPreferences.

يستخدم مفتاح الهوية للتوقيع والتحقق في المصافحة المستقبلية، وليس بديلاً مباشراً عن AES-GCM لتشفير كل محتوى التطبيق.

يجب وصف الحماية بدقة. Android Keystore يوفر حماية أقوى، وقد يعتمد على TEE أو StrongBox حسب الجهاز، لكنه ليس ضماناً مطلقاً ضد كل الهجمات. كما أن اختبارات Android Instrumented لم تُنفذ على جهاز أو محاكي حسب الحالة المسجلة.

8.5 Context-Bound Signature

في التصميم الأولي كان التوقيع يغطي المفتاح المؤقت فقط. تم تحسين ذلك ببناء بيانات توقيع موحدة وحتمية تشمل سياق المصافحة، مثل:

- إصدار البروتوكول.
- معرف المرسل.
- معرف المستقبل.
- معرف الجلسة.
- الطابع الزمني.
- المفتاح العام المؤقت.

يمنع ذلك إعادة استخدام التوقيع بسهولة في سياق مختلف، لأن تغيير أحد الحقول يؤدي إلى فشل التحقق. ولا ينبغي الخلط بين هذا الإجراء وبين Replay Protection الكاملة.

8.6 TOFU لتثبيت هوية القرين

تم إضافة **PeerTrustStore** لتخزين بصمة مفتاح هوية القرين. في أول اتصال تُحفظ البصمة وفق سياسة Trust On First Use. في الاتصالات اللاحقة تُقبل البصمة نفسها، ويُرفض تغييرها باعتباره تغييرًا في الهوية.

TOFU يحمي الاتصالات اللاحقة بعد تسجيل البصمة، لكنه لا يحمي أول اتصال بالكامل من MITM إذا لم يقارن المستخدم SAS أو بصمة عبر قناة خارجية. لذلك حالة MITM Protection هي **جزئية** وليست كاملة.

8.7 Session Key Scope و Handshake

تم دمج رسائل مصافحة في **LocalP2PEngine** وفق التقارير المرحلية، مع رسائل بدء ورد وتأكيد. ويستخدم Direct Chat مفتاح جلسة عند اكتمال المسار المناسب.

لكن نطاق استخدام Session Key ليس شاملًا لكل البيانات. الحالة الموثقة هي:

نوع البيانات	المفتاح الحالي	حالة الربط بالجلسة
Discovery	networkKey	غير مرتبط بجلسة
Direct Chat	sessionKey عند توفره، وإلا المسار القديم حيث ينطبق	جزئي
Room Chat	networkKey	غير مرتبط بجلسة
File Transfer	networkKey	غير مرتبط بجلسة
Audio	networkKey	غير مرتبط بجلسة
Video	networkKey	غير مرتبط بجلسة
Screen Sharing	networkKey	غير مرتبط بجلسة
Handshake	مسار مختلط أو Legacy bootstrap	جزئي

لهذا السبب لا يجوز وصف النظام بأنه يستخدم Session Key لكل أنواع البيانات.

9. مقارنة التشفير قبل وبعد

الجانب	قبل التحسين	بعد التحسين
تشفير البيانات	AES-256-GCM	AES-256-GCM ما زال الأساس
مصدر المفتاح	عبارة مرور ثابتة داخل التطبيق	ECDH / HKDF لمكونات مفاتيح الجلسات، مع بقاء Legacy في بعض المسارات
مفتاح عالمي مشترك	مستخدم في معظم المسارات	ما زال مستخدمًا في Discovery والغرف والملفات والوسائط حسب الحالة الموثقة
ECDH	غير موجود	مكون منفذ ومختبر على JVM
HKDF	غير موجود في التصميم القديم	مستخدم لاشتقاق Session Keys حسب المكون المنفذ
مفتاح لكل جلسة	غير موجود	موجود في مسار Direct Chat والمكونات المخصصة للجلسات

الجانب	قبل التحسين	بعد التحسين
هوية الجهاز	غير موجودة	Android Keystore و ECDSA حسب المكون المنفذ
توقيع المصافحة	غير موجود أو غير مرتبط بالسياق	Context-Bound Signature
تثبيت هوية القرين	غير موجود	TOFU
حماية MITM	غير موجودة	جزئية؛ SAS غير منفذة
حماية Downgrade	غير موجودة	جزئية؛ بعض المسارات تحتاج تدقيقًا إضافيًا
Replay Protection	غير موجودة	ما زالت غير مكتملة
File Integrity Checksum	غير موجود	ما زال غير منفذ
اختبار ميداني بين جهازين	غير موجود	ما زال غير منفذ

10. ما الذي تثبته نتائج الاختبار؟

تثبت نتيجة 33/33 أن الاختبارات التي تم تشغيلها نجحت على بيئة JVM. وتشمل النتيجة اختبارات AES-GCM و ECDH وإدارة الجلسات و Hardening وبعض اختبارات المشروع الأخرى.

لا تثبت النتيجة وحدها أن:

- الاتصال بين جهازين فعليين يعمل عبر Wi-Fi.
- Android Keystore تم التحقق منه على جهاز.
- MITM مستحيل.
- Replay Protection مكتملة.
- الملفات والصوت والفيديو والغرف تستخدم Session Keys.
- File Integrity Checksum منفذة.
- كل المتطلبات الوظيفية من FR-01 إلى FR-31 مختبرة.

11. ما تبقى خارج التنفيذ الكامل

الميزة	الحالة الحالية
SAS للتحقق اليدوي من الهوية	غير منفذة
Replay Protection حقيقية	غير منفذة ؛ لا يوجد sequence number وسجل رفض للمكرر
SHA-256 للملف الكامل	غير منفذ
تشفير الملفات بمفتاح جلسة	غير مكتمل ، وما زال المسار القديم مستخدمًا حسب التقرير
تشفير الصوت والفيديو بمفتاح جلسة	غير مكتمل
Group Key للغرف	غير منفذ
اختبار Android Instrumented	غير منفذ لعدم توفر جهاز أو محاكي
اختبار جهازين فعليين	غير منفذ
تغطية JaCoCo كاملة	غير مقاسة

12. مخرجات البناء

تم إنشاء نسخة Debug بنجاح:

BUILD SUCCESSFUL

ومسار ملف APK هو:

app/build/outputs/apk/debug/app-debug.apk

تم إنشاء debug.keystore محليًا لتوقيع نسخة Debug. هذا الملف مفتاح تطوير محلي، وليس مفتاح توقيع Production، ولا ينبغي إضافته إلى مستودع Git العام.

13. الملفات والوثائق الرئيسية

الملف أو المكون	الغرض
docs/SRS.md	مواصفات المتطلبات
docs/Baseline.md	خط أساس النظام
docs/TraceabilityMatrix.md	ربط المتطلبات بالكود والاختبارات
KeyProvider.kt	تجريد إدارة المفاتيح
LegacyStaticKeyProvider	الحفاظ على توافق المفتاح القديم مؤقتًا
KeystoreIdentityManager.kt	هوية الجهاز والتوقيع عبر Android Keystore
EcdhEngine.kt	ECDH والسر المشترك واشتقاق الجلسة
PeerSessionKeyManager.kt	إدارة مفاتيح وحالات جلسات الأقران
HandshakeCryptoUtils.kt	بناء بيانات التوقيع المرتبطة بسياق المصافحة
PeerTrustStore.kt	تخزين بصمة هوية القرين وفق TOFU
LocalP2PEngine.kt	دمج الاتصال والمصافحة في كود P2P
ملفات الاختبارات	إثبات سلوك المكونات المحددة

14. الخلاصة النهائية

قبل تطبيق مبادئ هندسة البرمجيات، كان المشروع أقرب إلى تطبيق يعمل وظيفيًا مع تشفير قوي على مستوى الخوارزمية، لكنه كان يفتقر إلى تنظيم المتطلبات والتتبع وإدارة المفاتيح الديناميكية والإثبات الميداني.

بعد التحسين، أصبح المشروع أكثر تنظيمًا وقابلية للصيانة والمناقشة. تم إنشاء وثائق رسمية، وفصل إدارة المفاتيح، وإضافة مكونات ECDH و HKDF و Android Keystore و TOFU و Context-Bound Signature، وتشغيل اختبارات JVM، وبناء APK قابل للتثبيت.

التحسين الأمني لا يعني أن كل مسارات التطبيق أصبحت محمية بمفتاح جلسة. ما زالت بعض المسارات تستخدم المفتاح الشبكي القديم، وما زالت Replay Protection و File Integrity Checksum و SAS والتحقق بين جهازين غير منفذة أو غير مثبتة. توثيق هذه الحدود هو جزء من هندسة البرمجيات الجيدة وليس نقطة ضعف في التقرير.

✦ الحكم النهائي:

المشروع مناسب للتسليم والمناقشة الأكاديمية، بشرط تقديمه على أنه نظام P2P موثق ومختبر على مستوى المكونات، مع وجود Handshake ومكونات تشفير متقدمة مدمجة، وليس على أنه نظام أمني نهائي تم التحقق منه ميدانيًا في جميع أنواع الاتصال.

النص الشفوي الجاهز للإلقاء أمام لجنة المناقشة:

"قبل التحسين كان التطبيق يعمل وظيفيًا، لكنه كان يعتمد على مفتاح تشفير ثابت مشترك، وكانت المتطلبات والاختبارات والفجوات غير مرتبطة بمصفوفة تتبع واضحة. بعد تطبيق مبادئ هندسة البرمجيات أنشأنا SRS و Baseline و Traceability Matrix، وفصلنا إدارة المفاتيح عبر KeyProvider، وأضفنا اختبارات مستقلة للمكونات. ومن ناحية التشفير حافظنا على AES-256-GCM، وأضفنا مكونات ECDH و HKDF لمفاتيح الجلسات، و Android Keystore لهوية الجهاز، و Context-Bound Signatures و TOFU. نجحت اختبارات JVM المسجلة بنتيجة 33 من 33، وتم بناء APK بنجاح. ومع ذلك، لم يتم اختبار جهازين فعليين، و SAS و Replay Protection الكاملة و File Integrity Checksum غير منفذة، وبعض مسارات الغرف والملفات والوسائط ما زالت تستخدم المفتاح القديم. لذلك نفرق في التقرير بين المكون المنفذ، والمكون المختبر، والمكون المدمج غير المختبر ميدانيًا، والمكون غير المنفذ."

References

- [1] [LOCAL CONTACT project commit reviewed for the baseline analysis](#)
- [2] [RFC 5869: HMAC-based Extract-and-Expand Key Derivation Function \(HKDF\)](#)
- [3] [NIST FIPS 197: Advanced Encryption Standard \(AES\)](#)
- [4] [Android Keystore system documentation](#)